

Chapter 4: Data Cube Computation and Data Generalization

- Efficient Computation of Data Cubes
- High-Dimensional OLAP Query
- Summary

Efficient Computation of Data Cubes

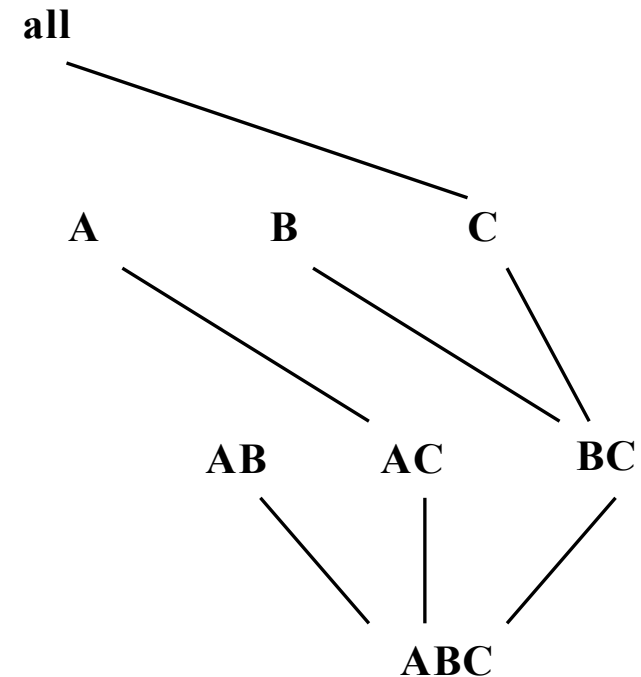
- Preliminary cube computation tricks
- Computing full/iceberg cubes:
 - Top-Down: **Multi-Way** array aggregation
 - Bottom-Up: **Bottom-up** computation: BUC
 - Integrating Top-Down and Bottom-Up:
- High-dimensional OLAP: A Minimal Cubing Approach.

Preliminary Tricks (Agarwal et al. VLDB'96)

- Sorting, hashing, and grouping operations are applied to the dimension attributes in order to reorder and cluster related tuples
- Aggregates may be computed from previously computed aggregates, rather than from **the base fact table**
 - **Smallest-child:** computing a cuboid from the smallest, previously computed cuboid
 - **Cache-results:** caching results of a cuboid from which other cuboids are computed to reduce disk I/Os
 - **Amortize-scans:** computing as many as possible cuboids at the same time to amortize disk reads
 - **Share-sorts:** sharing sorting costs across multiple cuboids when sort-based method is used
 - **Share-partitions:** sharing the partitioning cost across multiple cuboids when hash-based algorithms are used

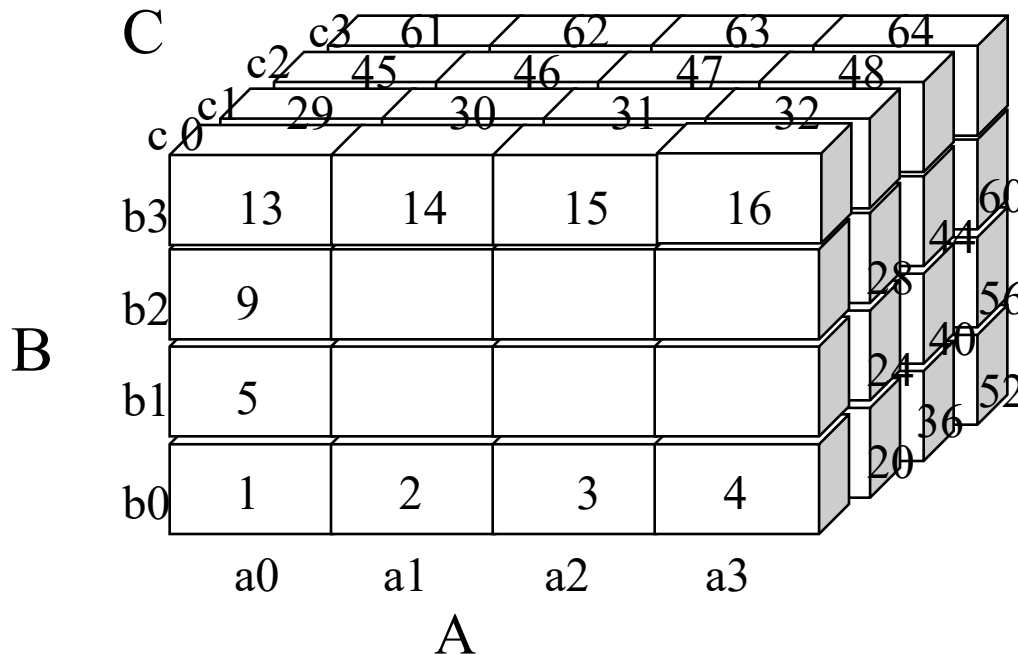
Multi-Way Array Aggregation

- Array-based “bottom-up” algorithm
- Using multi-dimensional chunks
- No direct tuple comparisons
- Simultaneous aggregation on multiple dimensions
- Intermediate aggregate values are re-used for computing ancestor cuboids



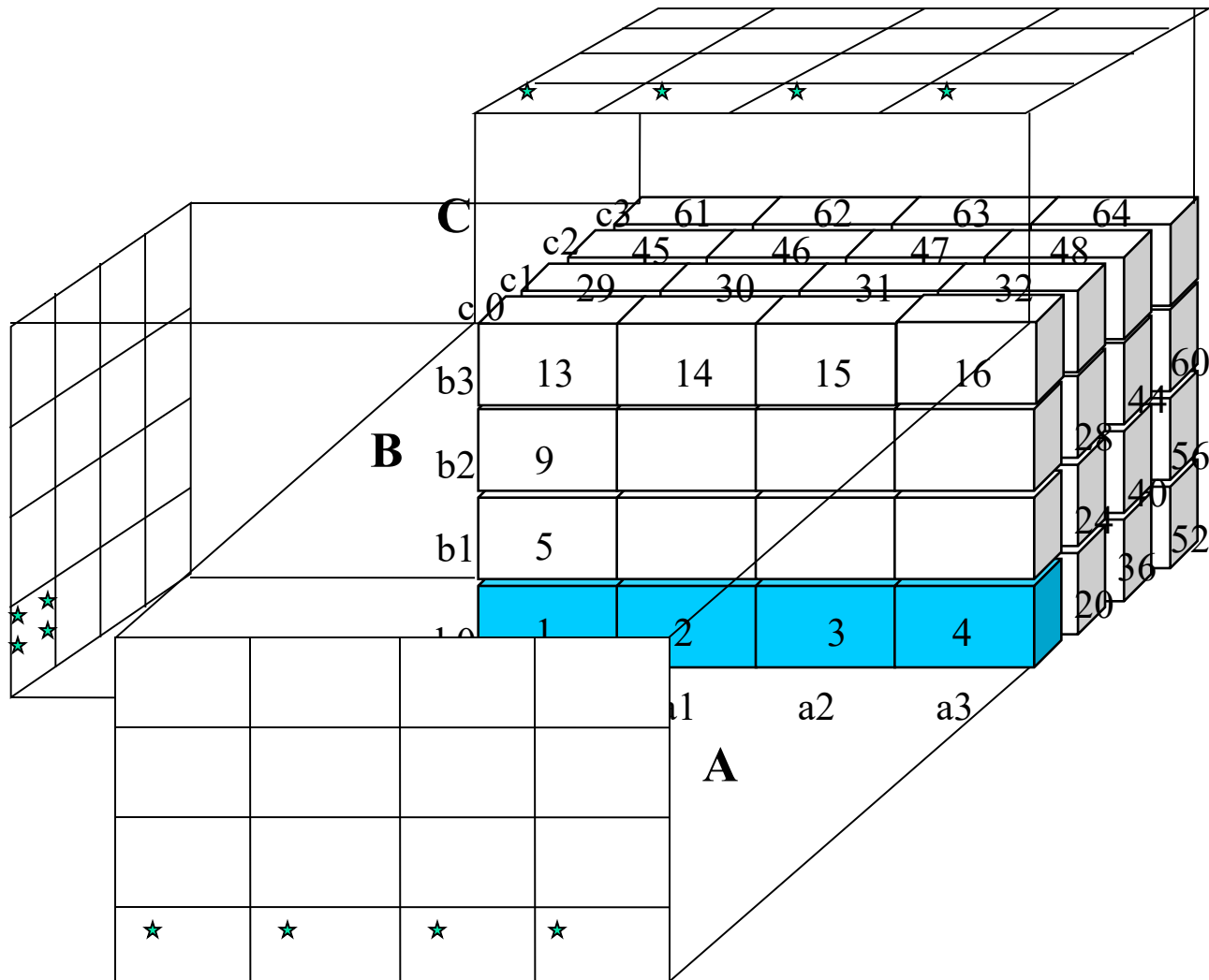
Multi-way Array Aggregation for Cube Computation (MOLAP)

- Partition arrays into **chunks** (a small subcube which fits in memory).
- Compressed sparse array addressing: (chunk_id, offset)
- Compute aggregates in “multiway” by visiting cube cells in the order which minimizes the # of times to visit each cell, and reduces memory access and storage cost.

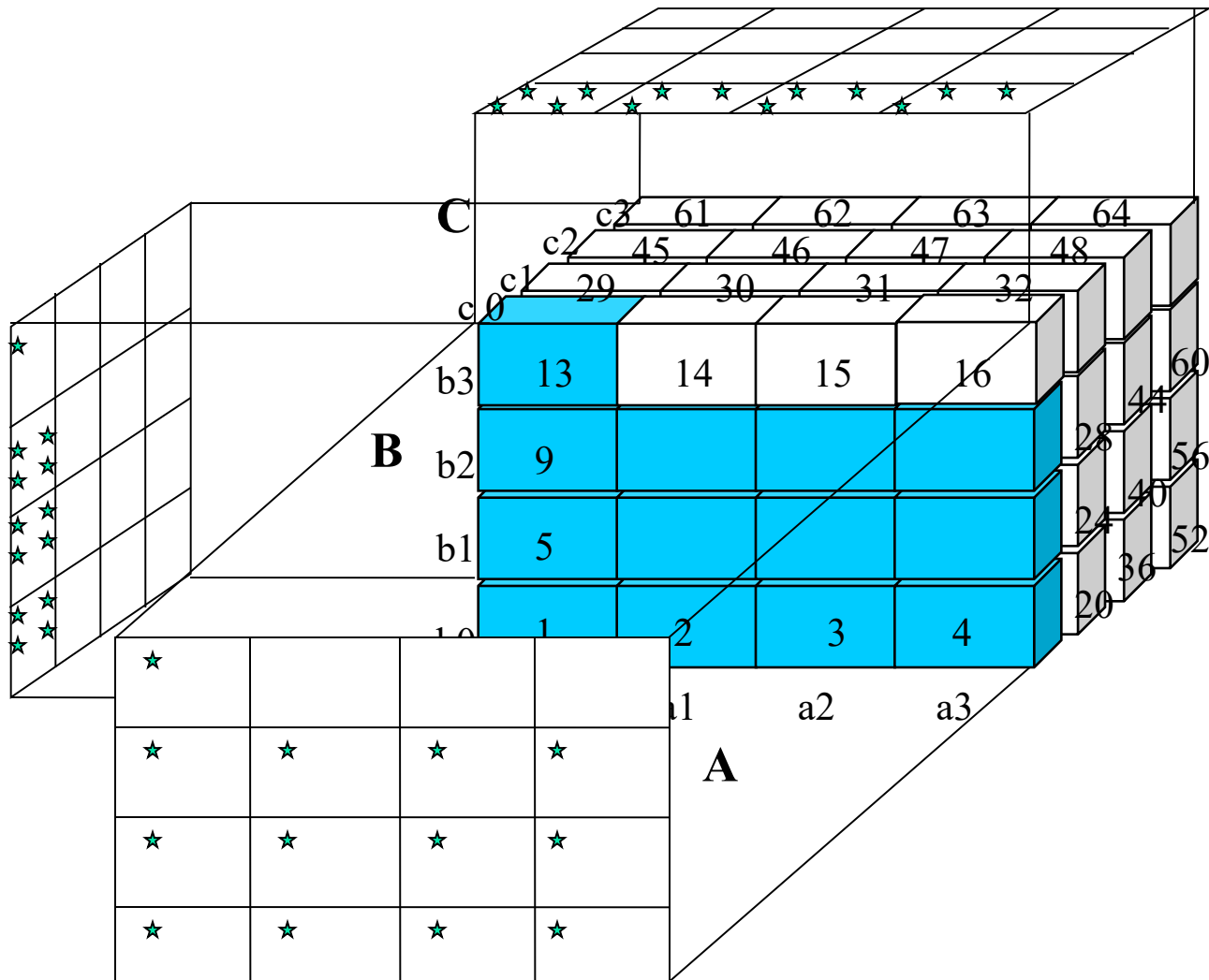


What is the best traversing order to do multi-way aggregation?

Multi-way Array Aggregation for Cube Computation



Multi-way Array Aggregation for Cube Computation

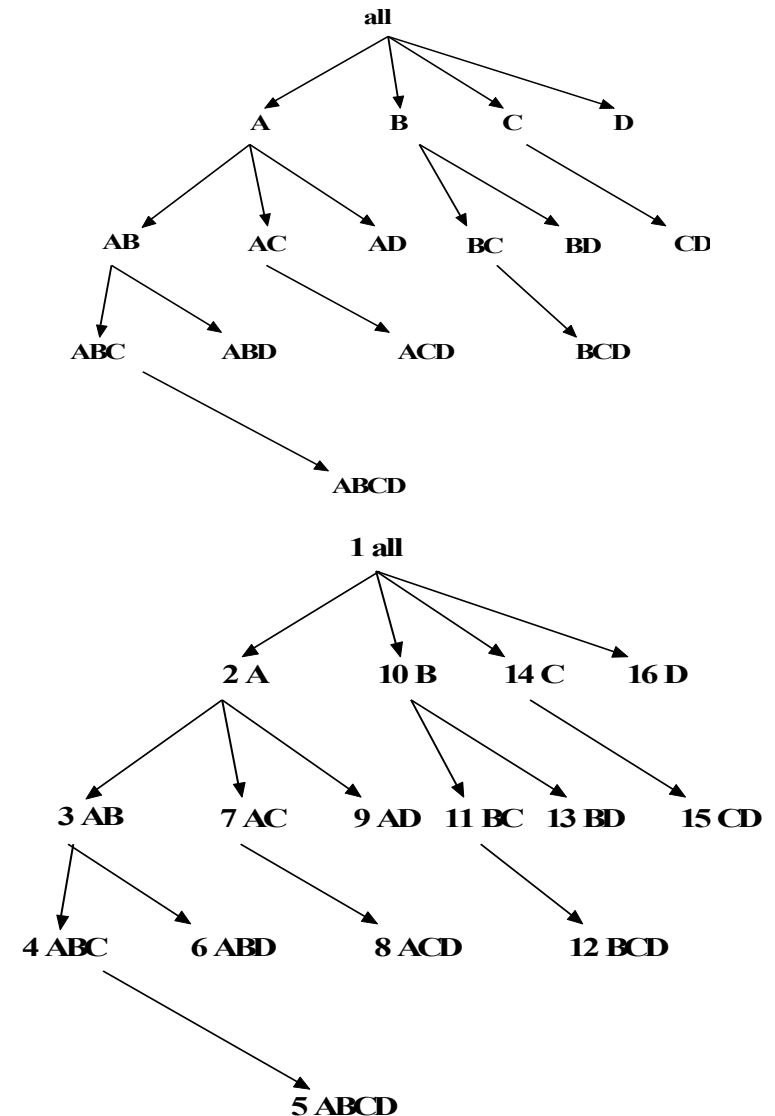


Multi-Way Array Aggregation for Cube Computation (Cont.)

- Method: the planes should be sorted and computed according to their size in ascending order
 - Idea: keep the smallest plane in the main memory, fetch and compute only one chunk at a time for the largest plane
- Limitation of the method: computing well only for a small number of dimensions
 - If there are a large number of dimensions, “top-down” computation and iceberg cube computation methods can be explored

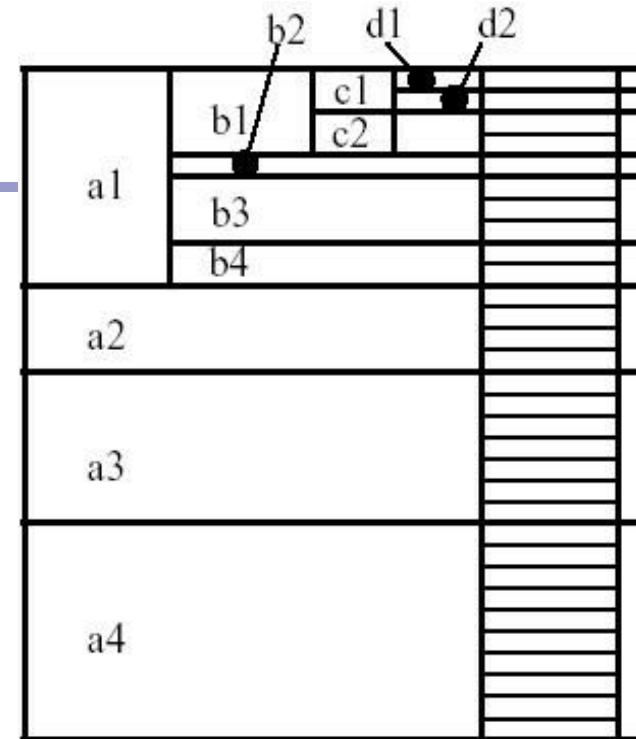
Bottom-Up Computation (BUC)

- Divides dimensions into segments and facilitates iceberg pruning
 - If a partition does not satisfy *min_sup*, its descendants can be pruned,
 - If *minsup* = 1 $\Rightarrow$ compute full CUBE!
- No simultaneous aggregation



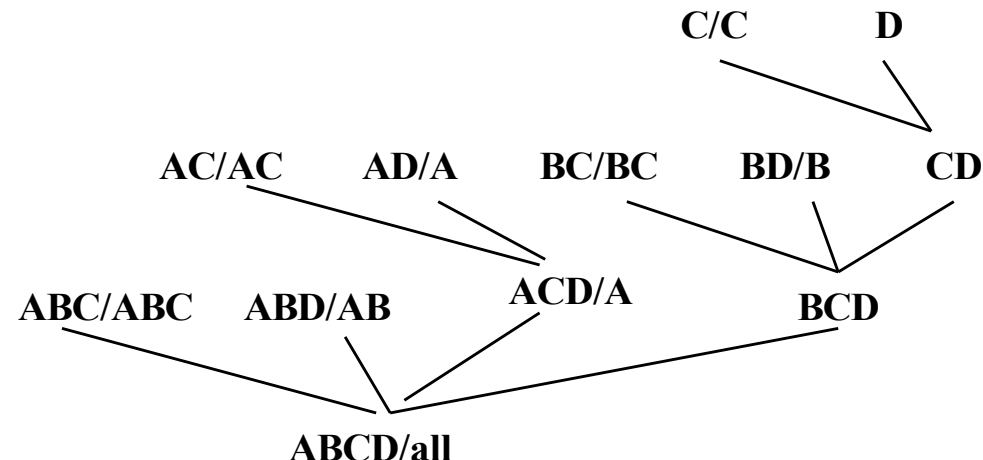
BUC: Partitioning

- Usually, entire data set can't fit in main memory
- Sort *distinct* values, partition into blocks that fit
- Continue processing
- Optimizations
 - Partitioning
 - External Sorting, Hashing, Counting Sort
 - Ordering dimensions to encourage pruning
 - Cardinality, Skew, Correlation
 - Collapsing duplicates
 - Can't do holistic aggregates anymore!



Star-Cubing: An Integrating Method

- Integrate the top-down and bottom-up methods
- Explore shared dimensions
 - E.g., dimension **A** is the shared dimension of **ACD** and **AD**
 - **ABD/AB** means cuboid **ABD** has shared dimensions **AB**
- Allows for shared computations
 - e.g., cuboid **AB** is computed simultaneously as **ABD**
- Aggregate in a top-down manner but with the bottom-up sub-layer underneath which will allow **Apriori** pruning
- Shared dimensions grow in bottom-up fashion



Iceberg Pruning in Shared Dimensions

- Anti-monotonic property of shared dimensions
 - If the measure is *anti-monotonic*, and if the aggregate value on a shared dimension does not satisfy the *iceberg condition*, then all the cells extended from this shared dimension cannot satisfy the condition either
- Intuition: if we can compute the shared dimensions before the actual cuboid, we can use them to do `Apriori` pruning
- Problem: how to prune while still aggregate simultaneously on multiple dimensions?

Star Attributes and Star Nodes

- Intuition: If a single-dimensional aggregate on an attribute value p does not satisfy the iceberg condition, it is useless to distinguish them during the iceberg computation
 - E.g., $b_2, b_3, b_4, c_1, c_2, c_4, d_1, d_2, d_3$
- Solution: Replace such attributes by a *. Such attributes are star attributes, and the corresponding nodes in the cell tree are star nodes

A	B	C	D	Count
a1	b1	c1	d1	1
a1	b1	c4	d3	1
a1	b2	c2	d2	1
a2	b3	c3	d4	1
a2	b4	c3	d4	1

Example: Star Reduction

- Suppose $\text{minsup} = 2$
- Perform one-dimensional aggregation. Replace attribute values whose count < 2 with *. And collapse all *'s together
- Resulting table has all such attributes replaced with the star-attribute
- With regards to the iceberg computation, this new table is a *loseless compression* of the original table

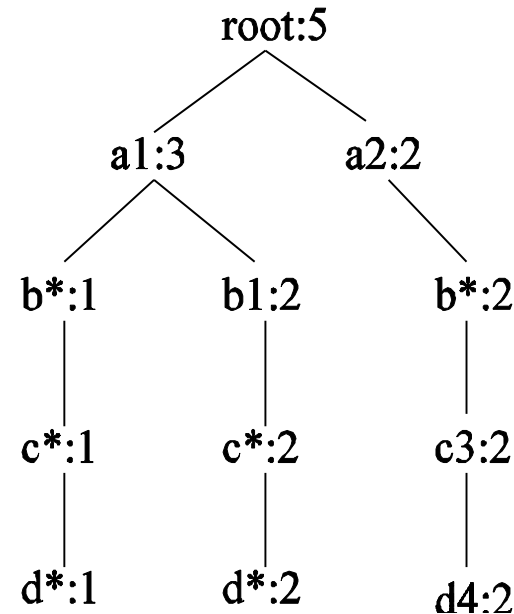
A	B	C	D	Count
a1	b1	*	*	1
a1	b1	*	*	1
a1	*	*	*	1
a2	*	c3	d4	1
a2	*	c3	d4	1



A	B	C	D	Count
a1	b1	*	*	2
a1	*	*	*	1
a2	*	c3	d4	2

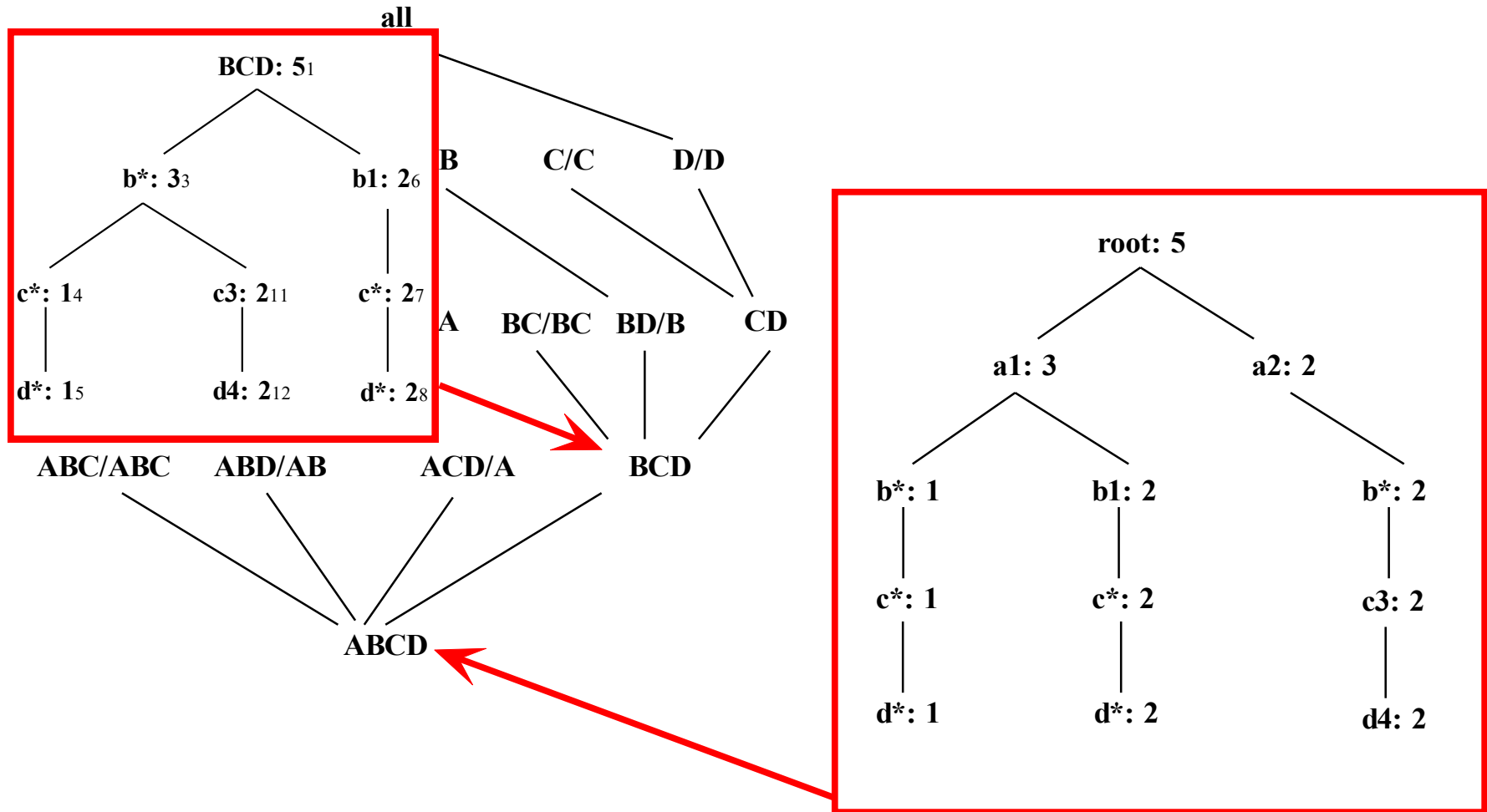
Star Tree

- Given the new compressed table, it is possible to construct the corresponding cell tree—called star tree
- Keep a star table at the side for easy lookup of star attributes
- The star tree is a *loseless compression* of the original cell tree

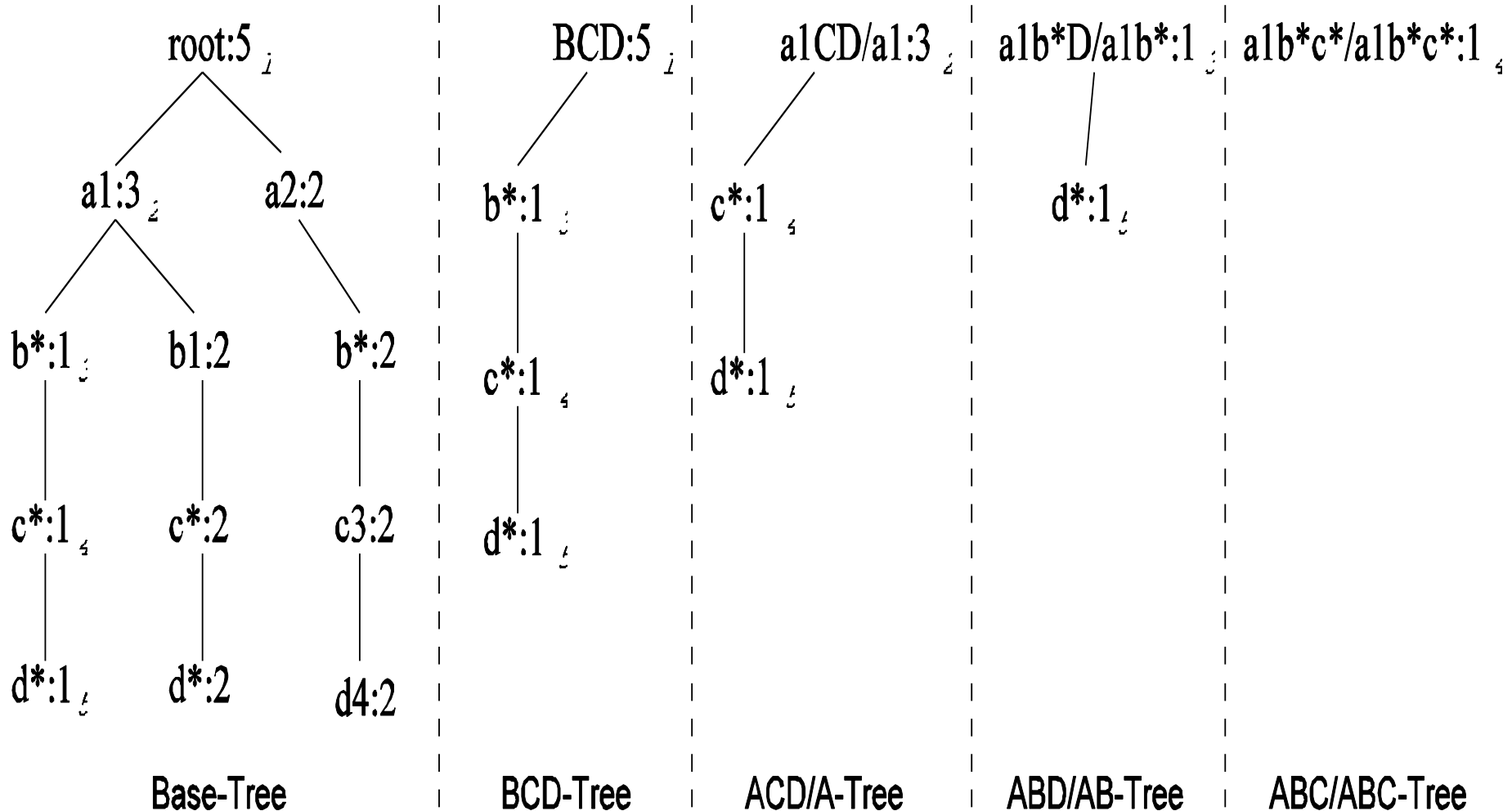
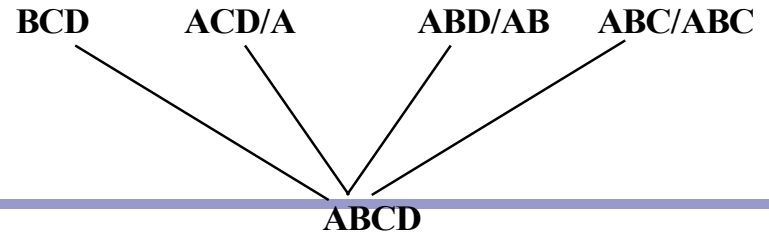


Star Table	
b2	→ *
b3	→ *
b4	→ *
c1	→ *
c2	→ *
d1	→ *
...	

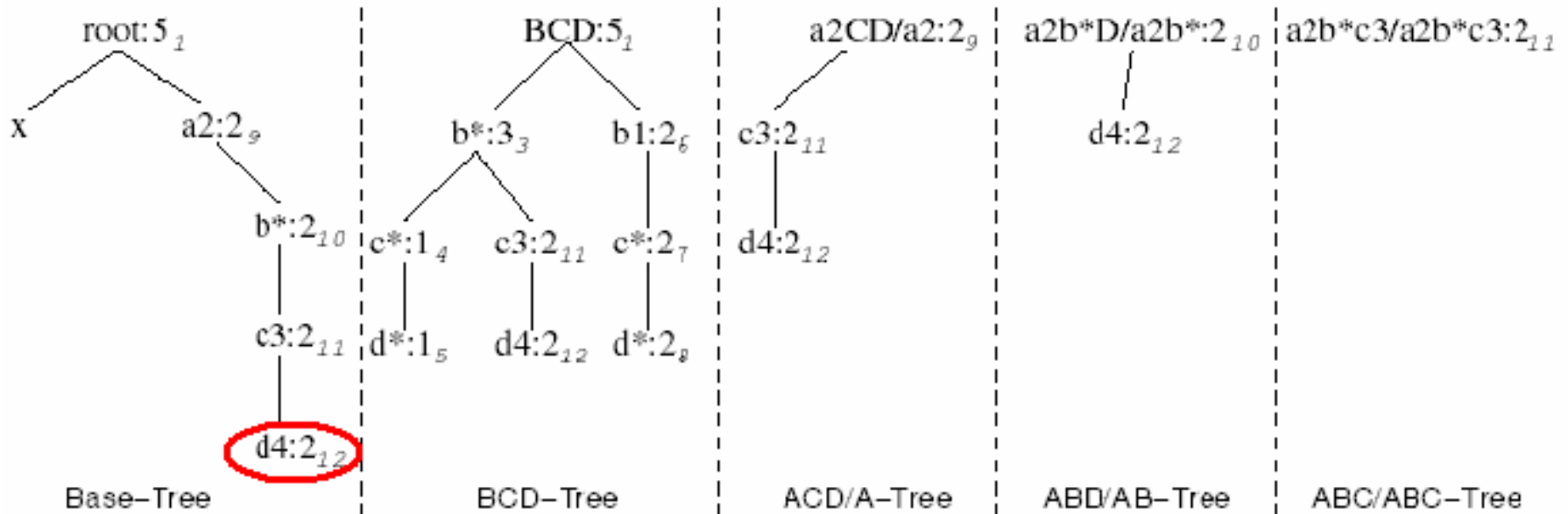
Star-Cubing Algorithm—DFS on Lattice Tree



Multi-Way Aggregation



Star-Cubing Algorithm—DFS on Star-Tree



Multi-Way Star-Tree Aggregation

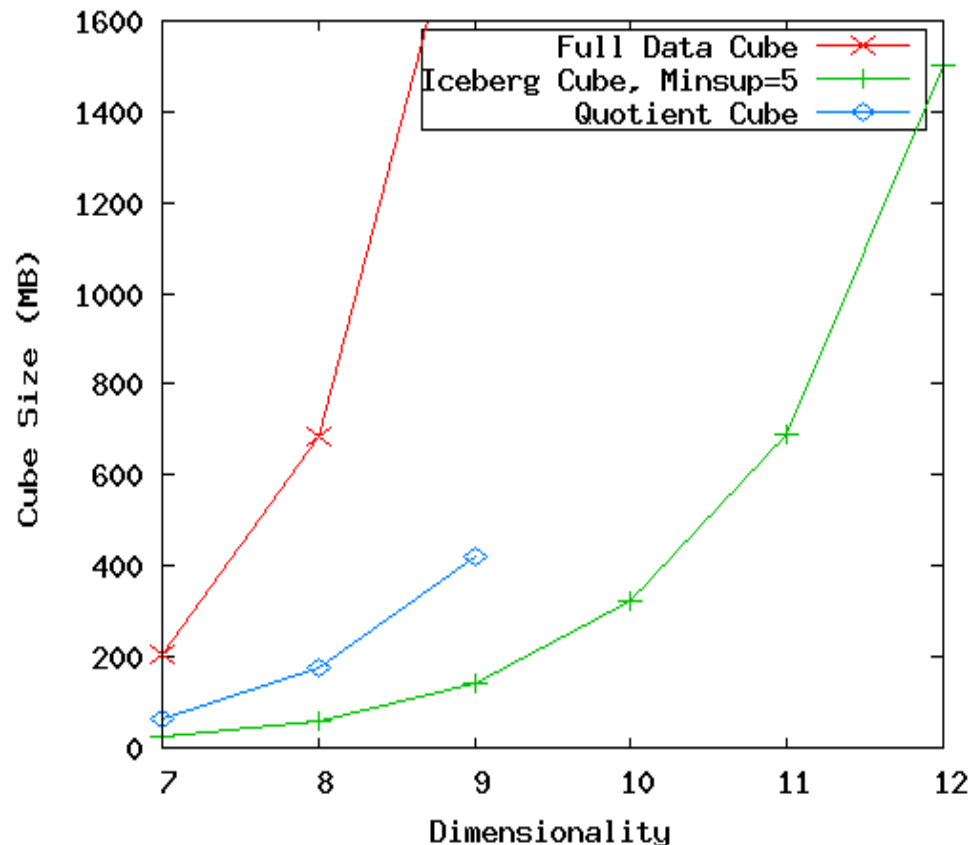
- Start depth-first search at the root of the base star tree
- At each new node in the DFS, create corresponding star tree that are descendants of the current tree according to the integrated traversal ordering
 - E.g., in the base tree, when DFS reaches a_1 , the ACD/A tree is created
 - When DFS reaches b^* , the ABD/AD tree is created
- The counts in the base tree are carried over to the new trees

Multi-Way Aggregation (2)

- When DFS reaches a leaf node (e.g., d^*), start backtracking
- On every backtracking branch, the count in the corresponding trees are output, the tree is destroyed, and the node in the base tree is destroyed
- Example
 - When traversing from d^* back to c^* , the $a1b^*c^*/a1b^*c^*$ tree is output and destroyed
 - When traversing from c^* back to b^* , the $a1b^*D/a1b^*$ tree is output and destroyed
 - When at b^* , jump to $b1$ and repeat similar process

The Curse of Dimensionality

- None of the previous cubing method can handle high dimensionality!
- A database of 600k tuples. Each dimension has cardinality of 100 and *zipf* of 2.



Motivation of High-D OLAP

- Challenge to current cubing methods:
 - The “curse of dimensionality” problem
 - Iceberg cube and compressed cubes: only delay the inevitable explosion
 - Full materialization: still significant overhead in accessing results on disk
- High-D OLAP is needed in applications
 - Science and engineering analysis
 - Bio-data analysis: thousands of genes
 - Statistical surveys: hundreds of variables

Fast High-D OLAP with Minimal Cubing

- Observation: OLAP occurs only on a small subset of dimensions at a time
- Semi-Online Computational Model
 - n Partition the set of dimensions into **shell fragments**
 - n Compute data cubes for each shell fragment while retaining **inverted indices** or **value-list indices**
 - n Given the pre-computed **fragment cubes**, dynamically compute cube cells of the high-dimensional data cube *online*

Properties of Proposed Method

- Partitions the data vertically
- Reduces high-dimensional cube into a set of lower dimensional cubes
- Online re-construction of original high-dimensional space
- Lossless reduction
- Offers ***tradeoffs*** between the amount of pre-processing and the speed of online computation

Example Computation

- Let the cube aggregation function be `count`

<i>tid</i>	A	B	C	D	E
1	a1	b1	c1	d1	e1
2	a1	b2	c1	d2	e1
3	a1	b2	c1	d1	e2
4	a2	b1	c1	d1	e2
5	a2	b1	c1	d1	e3

- Divide the 5 dimensions into 2 shell fragments:
 - (A, B, C) and (D, E)

1-D Inverted Indices

- Build traditional invert index or RID list

Attribute Value	TID List	List Size
a1	1 2 3	3
a2	4 5	2
b1	1 4 5	3
b2	2 3	2
c1	1 2 3 4 5	5
d1	1 3 4 5	4
d2	2	1
e1	1 2	2
e2	3 4	2
e3	5	1

Shell Fragment Cubes

- Generalize the 1-D inverted indices to multi-dimensional ones in the data cube sense

Cell	Intersection	TID List	List Size
a1 b1	1 2 3 $\cap$ 1 4 5	1	1
a1 b2	1 2 3 $\cap$ 2 3	2 3	2
a2 b1	4 5 $\cap$ 1 4 5	4 5	2
a2 b2	4 5 $\cap$ 2 3	$\otimes$	0

Shell Fragment Cubes (2)

- Compute all cuboids for data cubes ABC and DE while retaining the inverted indices
- For example, shell fragment cube ABC contains 7 cuboids:
 - A, B, C
 - AB, AC, BC
 - ABC
- This completes the offline computation stage

Shell Fragment Cubes (3)

- Given a database of T tuples, D dimensions, and F shell fragment size, the fragment cubes' space requirement is:

$$O\left(T \left\lceil \frac{D}{F} \right\rceil (2^F - 1)\right)$$

- For $F < 5$, the growth is sub-linear.

Shell Fragment Cubes (4)

- Shell fragments do not have to be disjoint
- Fragment groupings can be arbitrary to allow for maximum online performance
 - Known common combinations (e.g., <city, state>) should be grouped together.
- Shell fragment sizes can be adjusted for optimal balance between offline and online computation

ID_Measure Table

- If measures other than `count` are present, store in *ID_measure* table separate from the shell fragments

tid	count	sum
1	5	70
2	3	10
3	8	20
4	5	40
5	2	30

The Frag-Shells Algorithm

Partition set of dimension $(A_1, \dots, A_n)$ into a set of k fragments $(P_1, \dots, P_k)$.

Scan base table once and do the following

- insert $\langle \text{tid}, \text{measure} \rangle$ into ID_measure table.

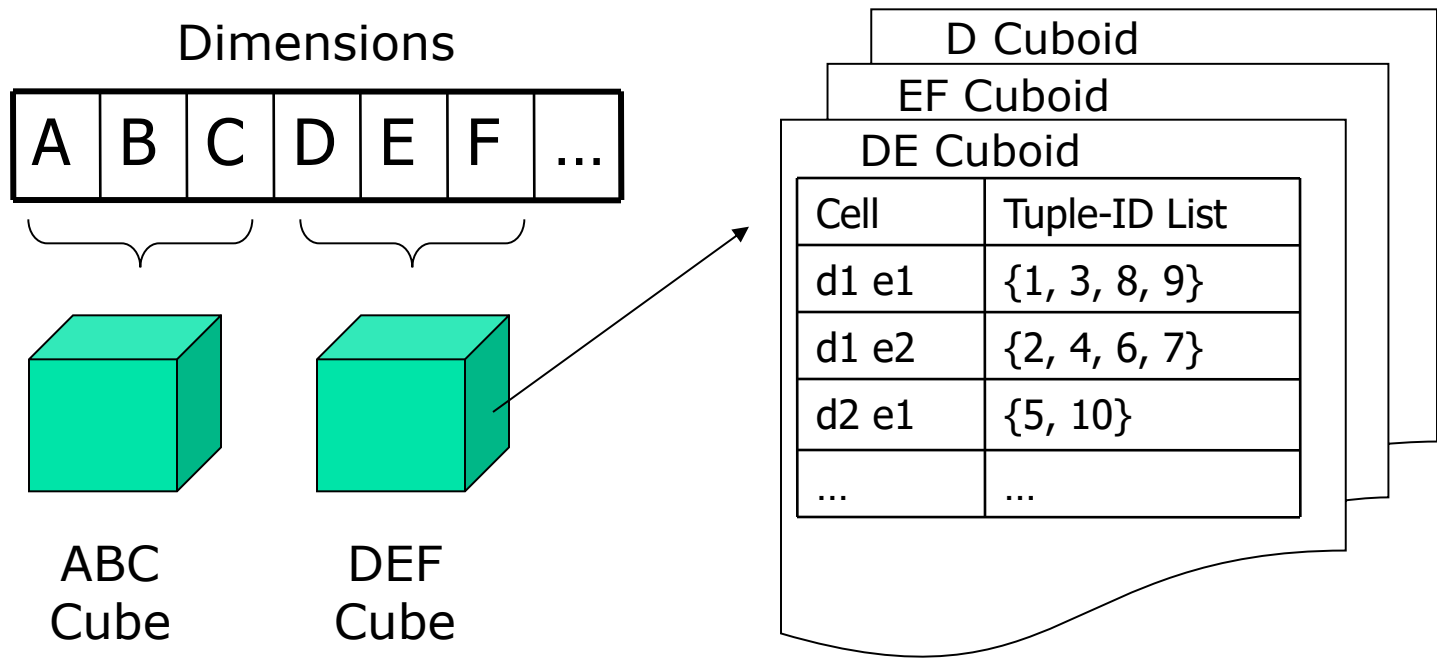
- for each attribute value a_i of each dimension A_i

 - build inverted index entry $\langle a_i, \text{tidlist} \rangle$

For each fragment partition P_i

- build local fragment cube S_i by intersecting tid-lists in bottom-up fashion.

Frag-Shells (2)



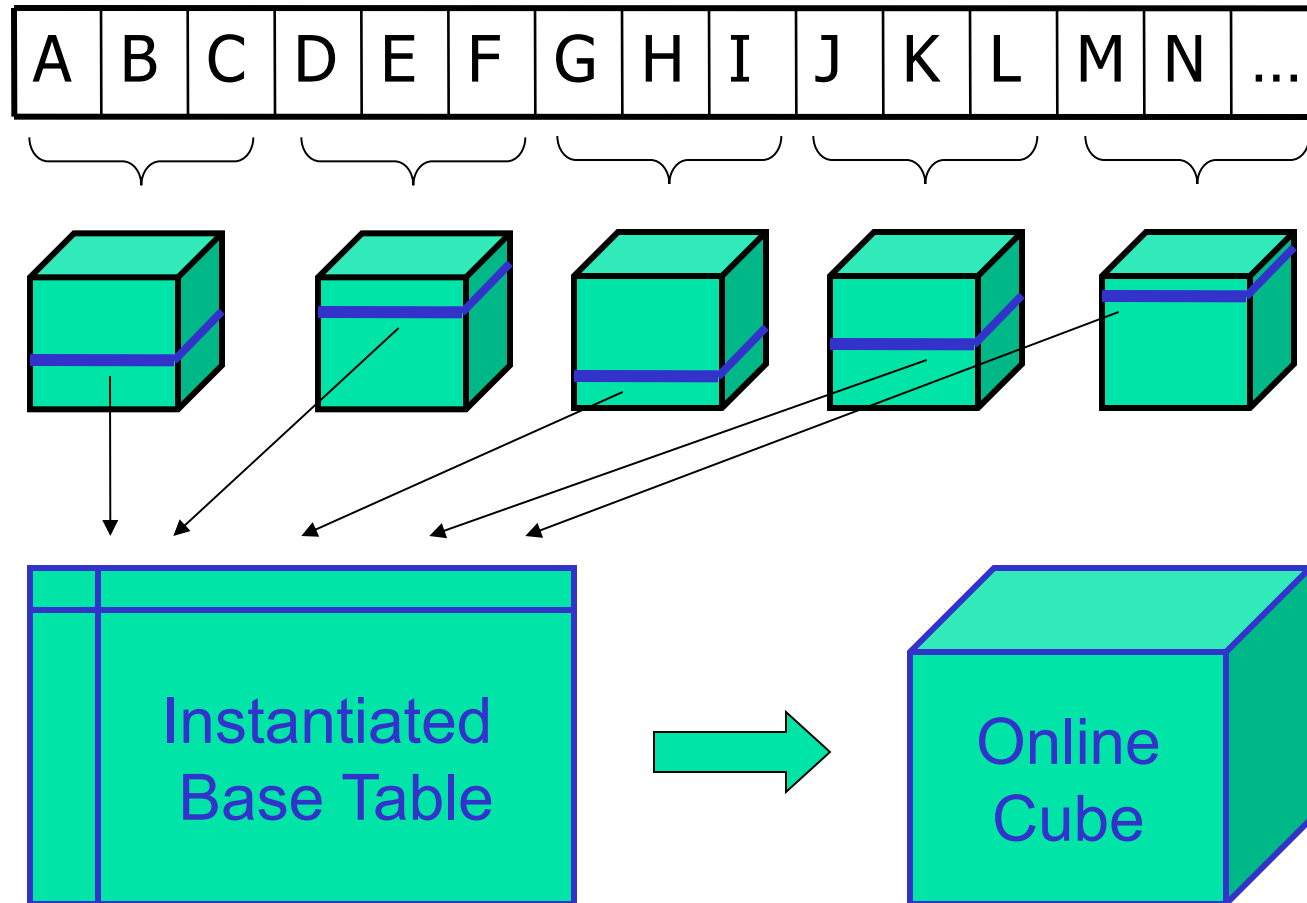
Online Query Computation

- A query has the general form $\langle a_1, a_2, \dots, a_n : M \rangle$
- Each a_i has 3 possible values
 1. Instantiated value
 2. Aggregate * function
 3. Inquire ? function
- For example, $\langle 3 \quad ? \quad ? \quad * \quad 1 : count \rangle$ returns a 2-D data cube.

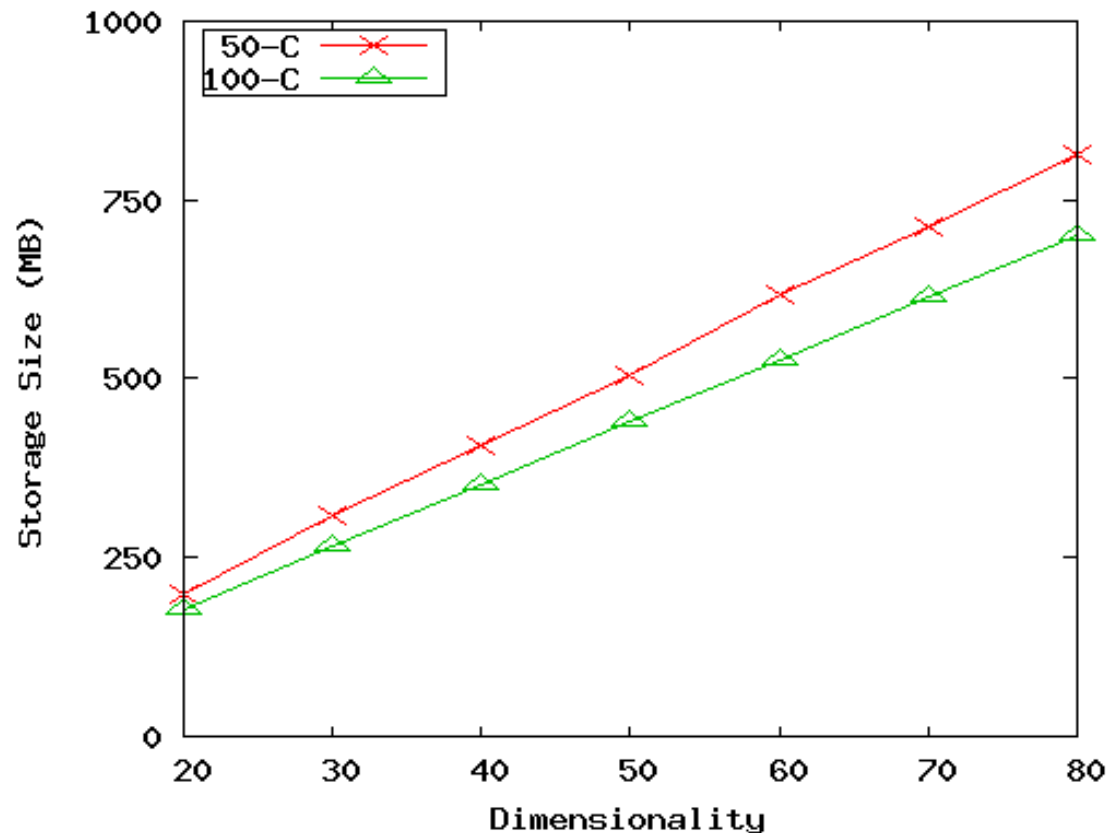
Online Query Computation (2)

- Given the fragment cubes, process a query as follows
 - n Divide the query into fragment, same as the shell
 - n Fetch the corresponding TID list for each fragment from the fragment cube
 - n Intersect the TID lists from each fragment to construct **instantiated base table**
 - n Compute the data cube using the base table with any cubing algorithm

Online Query Computation (3)

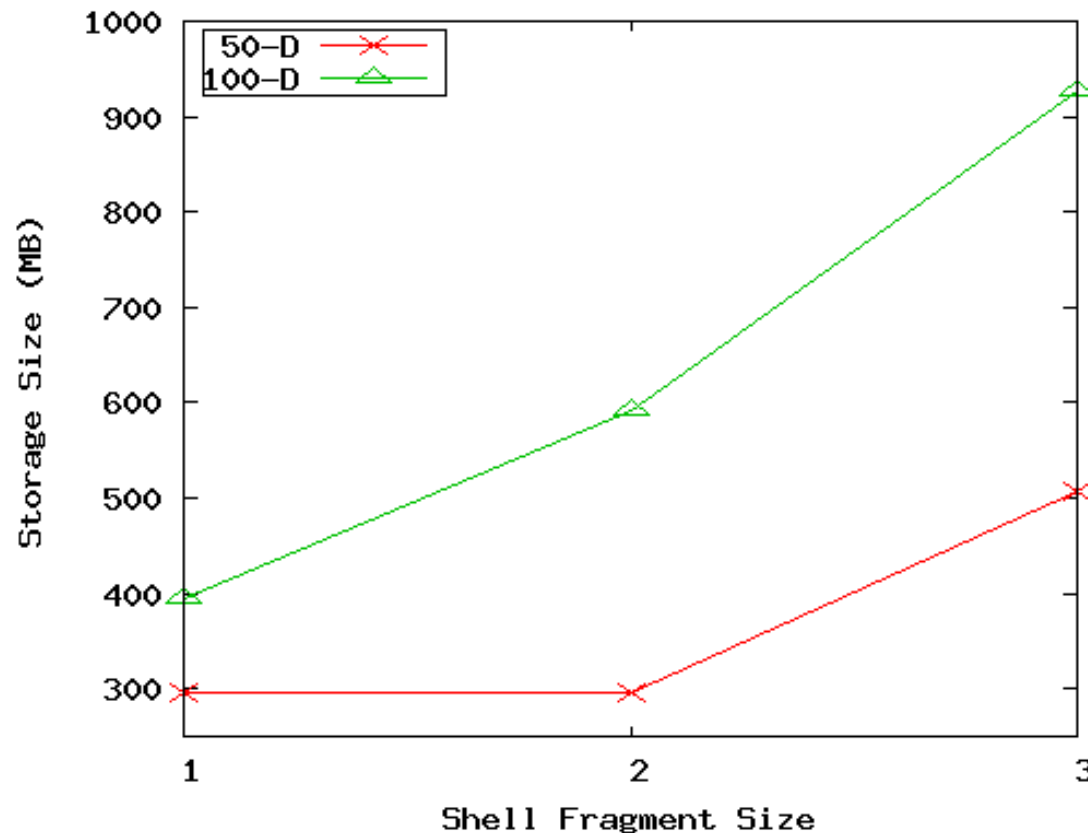


Experiment: Size vs. Dimensionality (50 and 100 cardinality)



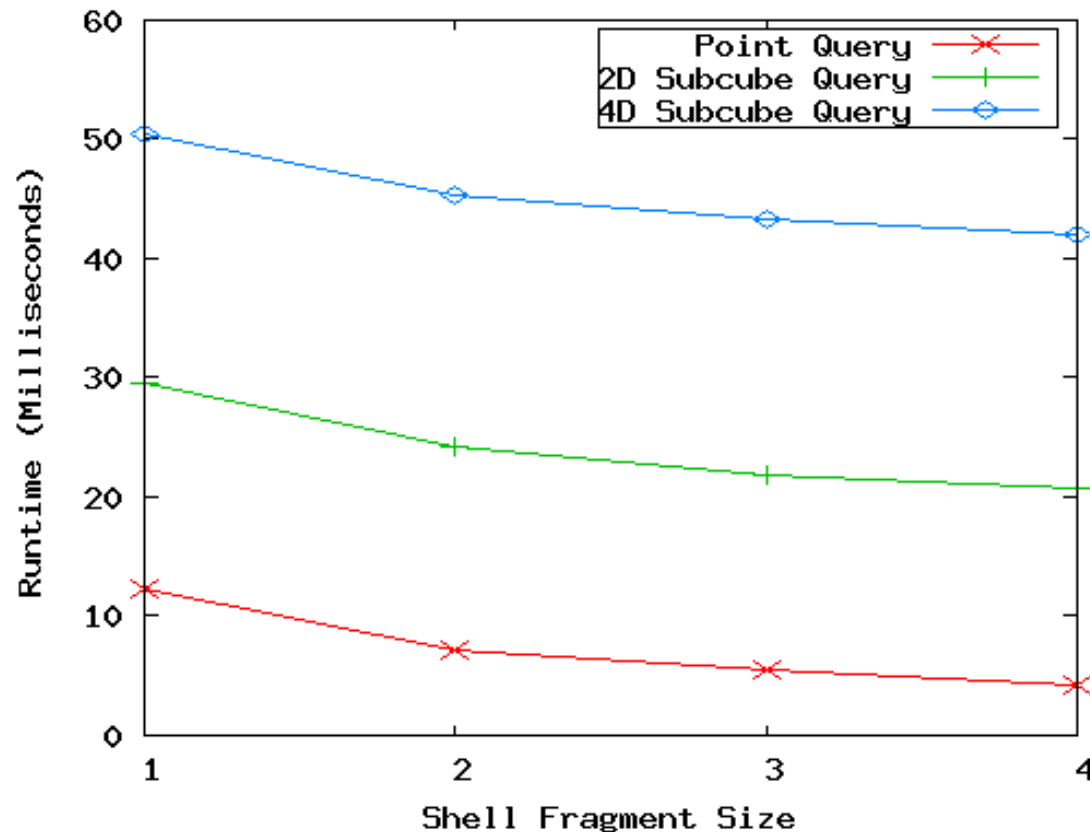
- (50-C): 10^6 tuples, 0 skew, 50 cardinality, fragment size 3.
- (100-C): 10^6 tuples, 2 skew, 100 cardinality, fragment size 2.

Experiment: Size vs. Shell-Fragment Size



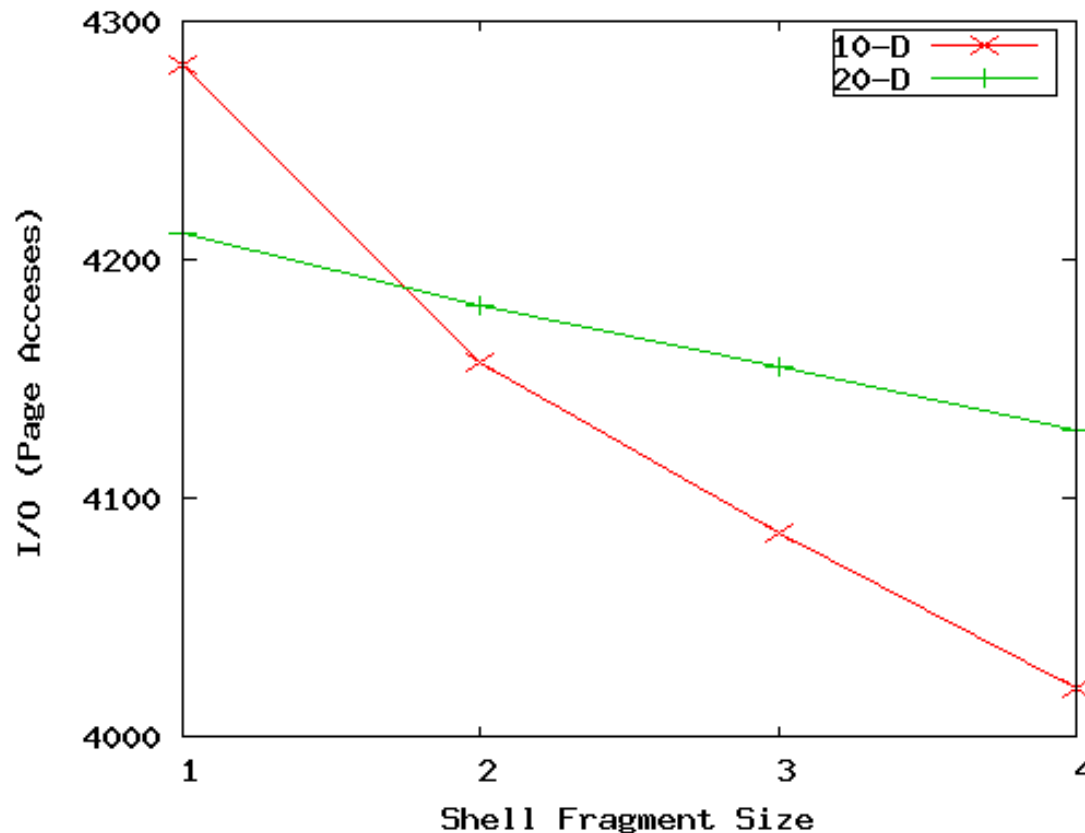
- (50-D): 10^6 tuples, 50 dimensions, 0 skew, 50 cardinality.
- (100-D): 10^6 tuples, 100 dimensions, 2 skew, 25 cardinality.

Experiment: Run-time vs. Shell-Fragment Size



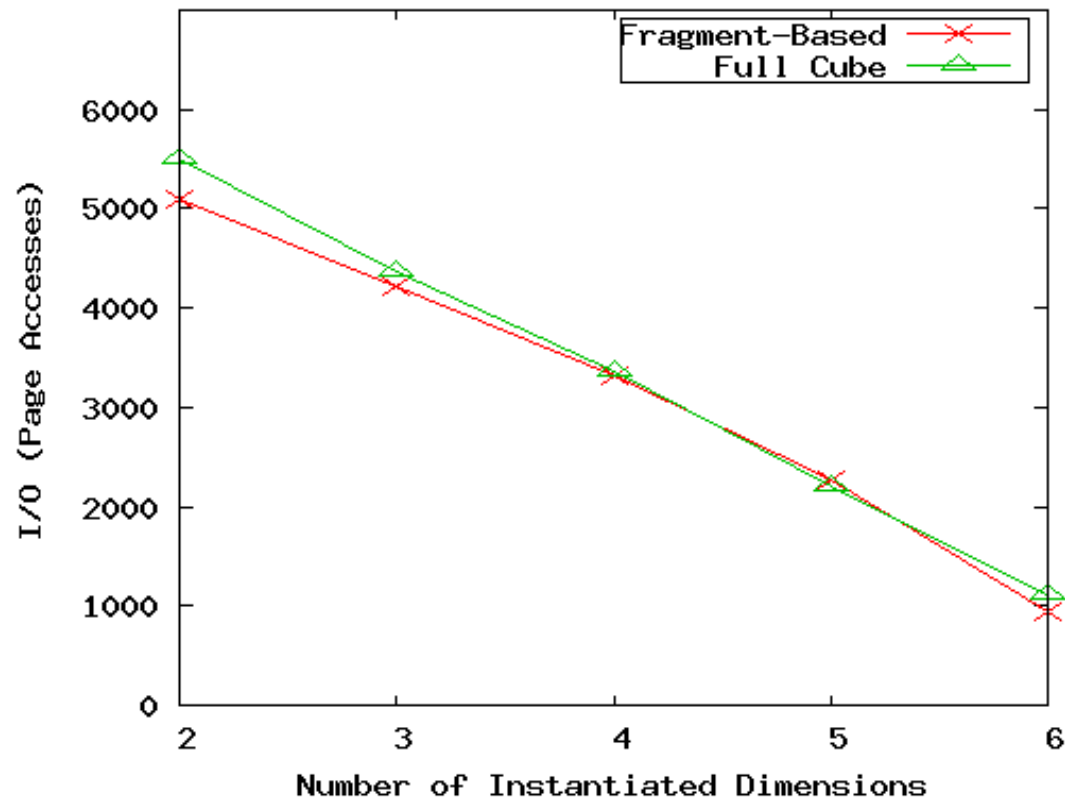
- 10^6 tuples, 20 dimensions, 10 cardinality, skew 1, fragment size 3, 3 instantiated dimensions.

Experiment: I/O vs. Shell-Fragment Size



- (10-D): 10^6 tuples, 10 dimensions, 10 cardinality, 0 skew, 4 inst., 4 query.
- (20-D): 10^6 tuples, 20 dimensions, 10 cardinality, 1 skew, 3 inst., 4 query.

Experiment: I/O vs. # of Instantiated Dimensions



- 10^6 tuples, 10 dimensions, 10 cardinality, 0 skew, fragment size 1, 7 total relevant dimensions.

Summary

- Efficient algorithms for computing data cubes
 - Multiway array aggregation
 - BUC
 - Star-cubing
 - High-D OLAP by minimal cubing

References (I)

- S. Agarwal, R. Agrawal, P. M. Deshpande, A. Gupta, J. F. Naughton, R. Ramakrishnan, and S. Sarawagi. On the computation of multidimensional aggregates. VLDB'96
- D. Agrawal, A. E. Abbadi, A. Singh, and T. Yurek. Efficient view maintenance in data warehouses. SIGMOD'97
- R. Agrawal, A. Gupta, and S. Sarawagi. Modeling multidimensional databases. ICDE'97
- K. Beyer and R. Ramakrishnan. Bottom-Up Computation of Sparse and Iceberg CUBEs.. SIGMOD'99
- Y. Chen, G. Dong, J. Han, B. W. Wah, and J. Wang, Multi-Dimensional Regression Analysis of Time-Series Data Streams, VLDB'02
- G. Dong, J. Han, J. Lam, J. Pei, K. Wang. Mining Multi-dimensional Constrained Gradients in Data Cubes. VLDB' 01
- J. Han, Y. Cai and N. Cercone, Knowledge Discovery in Databases: An Attribute-Oriented Approach, VLDB'92
- J. Han, J. Pei, G. Dong, K. Wang. Efficient Computation of Iceberg Cubes With Complex Measures. SIGMOD'01

References (II)

- L. V. S. Lakshmanan, J. Pei, and J. Han, Quotient Cube: How to Summarize the Semantics of a Data Cube, VLDB'02
- X. Li, J. Han, and H. Gonzalez, High-Dimensional OLAP: A Minimal Cubing Approach, VLDB'04
- K. Ross and D. Srivastava. Fast computation of sparse datacubes. VLDB'97
- K. A. Ross, D. Srivastava, and D. Chatziantoniou. Complex aggregation at multiple granularities. EDBT'98
- S. Sarawagi, R. Agrawal, and N. Megiddo. Discovery-driven exploration of OLAP data cubes. EDBT'98
- G. Sathe and S. Sarawagi. Intelligent Rollups in Multidimensional OLAP Data. VLDB'01
- D. Xin, J. Han, X. Li, B. W. Wah, Star-Cubing: Computing Iceberg Cubes by Top-Down and Bottom-Up Integration, VLDB'03
- D. Xin, J. Han, Z. Shao, H. Liu, C-Cubing: Efficient Computation of Closed Cubes by Aggregation-Based Checking, ICDE'06
- W. Wang, H. Lu, J. Feng, J. X. Yu, Condensed Cube: An Effective Approach to Reducing Data Cube Size. ICDE'02
- Y. Zhao, P. M. Deshpande, and J. F. Naughton. An array-based algorithm for simultaneous multidimensional aggregates. SIGMOD'97